MILLS' CONSTANT AND NUMERICAL COMPUTATION IN PYTHON.

Hammad A. Usmani
hammadus@gmail.com

ABSTRACT

Mills' Constant is part of a formula that generates primes and can be used to test the limits and edge cases of computation. In this work, we compute 1,655,460 digits of the Mills' Constant and publicly provide the software to be available for reproduction by assuming the Riemann Hypothesis. The problems arising from computing Mills' Constant include operations with both very large and very small numbers. In the source code, there are methods for numerical computation in Python that build on top of included capabilities. For example, floating point operations in Python have fixed precision by default, and this work achieves arbitrarily large precision limited only by available memory. In addition, this framework consists of optimizations that have produced a 98.41% reduction in compute time. Further applications can include enhancing the determinism of Large Language Models' temperature parameters through floating-point arithmetic.

***K*eywords** Mills' Constant · Arbitrary-Precision Arithmetic · Prime Number Generation · Catastrophic Cancellation · Python Performance · LLM Determinism

1 Introduction

In 1947, William H. Mills proved a fundamental theorem in number theory showing the existence of a real number θ such that the floor sequence $\lfloor \theta^{3^n} \rfloor$ yields a prime number for all positive integers n [1]. This expression serves as a popular classic mathematical framework for prime generation pipelines [11]. However, evaluating the precise fractional composition of θ is computationally difficult. The underlying prime sequences scale at a severe triple-exponential rate where subsequent terms quickly grow to span millions of individual digits [2].

To compute these values, implementations must analyze known prime-gap parameters while assuming the validity of the Riemann Hypothesis to guarantee stable sequence spacing boundaries [2]. Historical benchmarks mapped early index subsets, but recent breakthroughs identifying higher sequence offsets ($a(14) = 8436308$) by Propper and Batalov [3, 6] provided the necessary structural criteria to calculate the constant significantly beyond the 555,153-digit boundaries reached in previous open replication experiments [5].

This note introduces a Python framework designed to push this computational frontier by resolving exactly 1,665,460 decimal digits of the constant. The open-source replication environment is fully documented and hosted for public distribution [4].

2 Mathematical Formulation

Rather than using forward exponentiation, calculating θ involves determining a sequence of tightening bounds using known Mills primes (P_n). The constant can be represented as a limit of these prime configurations:

$$\theta = \lim_{n \to \infty} P_n^{3^{-n}} \tag{1}$$

This allows us to evaluate the value within structured, narrowing intervals defined by:

$$P_n^{3^{-n}} \leq \theta < (P_n + 1)^{3^{-n}} \tag{2}$$

The framework programmatically processes these limits by parsing sequence components through targeted database indexes.

```python
import pandas as pd

# Load sequence coordinates
df = pd.read_csv('mills-primes.csv')

# Isolate the target prime index configuration
n = 15
prime = df[df["n"] == n]["prime"].iloc
prime = int(prime) # Convert to an arbitrary precision integer
```

Figure 1: Programmatic parsing structure for target Mills primes.

3 Software Architecture and Precision Guarding

Native programming language architectures evaluate floating-point expressions with fixed binary precision (e.g., standard IEEE 754 64-bit definitions), causing rounding errors when processing deep decimal strings.

3.1 Arbitrary Precision Scaling

To bypass these hardware constraints, this software utilizes the multi-precision library `mpmath` [7]. While standard pure-Python floating-point emulation introduces structural processing loops, this setup integrates `mpmath` with the compiled C-based `gmpy2` dependency [9]. Linking directly with the GNU Multiple Precision Arithmetic Library (GMP) optimizes the mathematical overhead, yielding a 98.41% reduction in total compute time relative to unoptimized implementations.

3.2 Catastrophic Cancellation Guard

An unexpected edge case involving catastrophic cancellation occurs during the execution loop of calculating the n^{th} root of a targeted prime string [8]. Because `mpmath` manages internal mathematical states in a binary format, native boundaries experience long strings of consecutive zero bits for floating-point calculations at hyper-extended configurations.

When transforming these raw binary layouts into base-10 (decimal strings), a conversion shift occurs. The mathematical conversion steps can drop the primary ones-place integer by 1 unit, introducing an endless sequence of trailing 9s in subsequent decimal fractions (e.g., yielding $\ldots.999999\ldots$ instead of the exact rounded representation). During data validation phases, our code includes specific rounding guards to intercept boundary drops and align the base-10 text output with the physical values outlined in `mpmath` reference specifications.

4 Downstream Applications: LLM Determinism

Beyond standard number theory exploration, the high-precision arithmetic paradigms evaluated by this project provide immediate architectural insights for artificial intelligence optimization. In large language model (LLM) serving frameworks, temperature adjustments scale logit distributions prior to applying a Softmax function:

$$S_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{3}$$

When the temperature parameter T approaches near-zero values ($\epsilon \to 0$) to generate deterministic greedy selections, standard fixed-precision hardware frequently triggers underflow errors or catastrophic floating-point cancellations. Applying the arbitrary-precision constraints and radix truncation guards explored in this repository directly allows neural network execution environments to maintain predictable logit selection matrices, ensuring deterministic software output at extreme scaling bounds.

5 Conclusion

This framework successfully computes and preserves a 1,665,460-digit representation of Mills' constant. By detailing the interactions between arbitrary-precision scaling environments and internal conversion loops, this research provides an open, optimized testing environment for multi-precision computing and high-performance number theory verification.

References

[1] W. H. Mills. A prime-generating function. *Bulletin of the American Mathematical Society*, 53(6):604–604, 1947.

[2] C. K. Caldwell and Y. Cheng. Determining Mills' Constant and a Note on Honaker's Problem. *Journal of Integer Sequences*, 8(4):Article 05.4.1, 2005. https://cs.uwaterloo.ca/journals/JIS/VOL8/Caldwell/caldwell78.pdf

[3] OEIS Foundation Inc. Sequence A108739: $a(n) = b(n+1) - b(n)^3$, where $b(n)$ is the sequence of Mills primes. *The On-Line Encyclopedia of Integer Sequences*. https://oeis.org/A108739

[4] H. A. Usmani. Mills-Constant: 1,665,460 Digits of the Mills Constant. *GitHub Repository*, 2026. https://github.com/hammad93/mills-constant/

[5] ZabeMath. 555,153 Digits of the Mills Constant. *GitHub Repository*, 2019. https://github.com/ZabeMath/mills-constant

[6] S. Batalov. Mills Primes (PRPs) series extensions. *GitHub Repository*, 2023. https://github.com/s-batalov/Mills_Primes

[7] F. Johansson et al. mpmath: a Python library for arbitrary-precision floating-point arithmetic. Version 1.3.0. https://mpmath.readthedocs.io/en/1.3.0/index.html

[8] mpmath Documentation. Powers and Roots (nth root). https://mpmath.readthedocs.io/en/1.3.0/functions/powers.html#root

[9] mpmath Documentation. Using gmpy (optional). https://mpmath.readthedocs.io/en/1.3.0/setup.html#using-gmpy-optional

[10] Wikipedia contributors. Mills' constant. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Mills'_constant

[11] Numberphile. Awesome Prime Number Constant (Mills' Constant). *YouTube*, featuring James Grime, 2013. https://www.youtube.com/watch?v=6ltrPVPEwfo